

TASK 8: #51621

GOAL: automate the start/stop of ec2 instances using lambda and cloudwatch services.

- 1.create required IAM policies and Roles.
- 2.create function under aws-lambda which determines the start/stop process of specific ec2-instance.
- 3.test the function created under lambda
- 4.create rules inside cloudwatch service which trigger the function created in lambda.{ cron }

4. Step 1 — Create the IAM Policy

Lambda requires permission to start and stop the target EC2 instance. Create a dedicated IAM policy with only the required EC2 permissions.

Navigate to: AWS Console → IAM → Policies → Create policy

Select the JSON editor and use:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs::*:*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "ec2:Start*",
        "ec2:Stop*"
      ],
      "Resource": "*"
    }
  ]
}
```

```
}  
  
]  
  
}
```

Replace REGION, ACCOUNT_ID, and INSTANCE_ID with the actual values. For example, for ap-south-1 and instance i-0123456789abcdef0:

Policy name:mylambda

Click Create policy.

5. Step 2 — Create the IAM Role

Create an execution role that Lambda can assume. Attach the EC2 policy created above and the standard Lambda logging policy.

1. Go to AWS Console → IAM → Roles → Create role.
2. Select AWS service as the trusted entity.
3. Select Lambda as the service/use case.
4. Attach mylambda.
5. Attach AWSLambdaBasicExecutionRole so Lambda can write logs to CloudWatch Logs.
6. Name the role EC2StartStopLambdaRole.
7. Create the role.

Permission flow: Lambda function → assumes IAM role → role has IAM policy → policy permits EC2 operations.

6. Step 3 — Create the Lambda Function

Navigate to: AWS Console → Lambda → Functions → Create function

8. Choose Author from scratch.
9. Function names: start and stop
10. Choose a current supported Python 3.x runtime.
11. Choose x86_64 architecture unless your environment requires another architecture.
12. Under permissions, select Use an existing role.
13. Select Start .
14. Click Create function.
15. And create one more for stop function using stop role

7. Lambda Function Code

START code:

The function reads an action from the event. The action can be start or stop.

```
import boto3  
  
region = 'us-west-1'
```

```
instances = ['i-12345cb6de4f78g9h', 'i-08ce9b2d7eccf6d26']

ec2 = boto3.client('ec2', region_name=region)

def lambda_handler(event, context):

    ec2.start_instances(InstanceIds=instances)

    print('started your instances: ' + str(instances))
```

Replace INSTANCE_ID with the actual EC2 instance ID, then click Deploy.

Second function stop:

STOP code:

```
import boto3

region = 'us-west-1'

instances = ['i-12345cb6de4f78g9h', 'i-08ce9b2d7eccf6d26']

ec2 = boto3.client('ec2', region_name=region)

def lambda_handler(event, context):

    ec2.stop_instances(InstanceIds=instances)

    print('stopped your instances: ' + str(instances))
```

Replace INSTANCE_ID with the actual EC2 instance ID, then click Deploy.

8. Step 4 — Test the Lambda Function

Before creating schedules, test Lambda manually.

Create a test event named StartEC2Test and click Test. The EC2 instance should move from Stopped to Pending and then Running.

Create a test event named StopEC2Test and click Test. The EC2 instance should move from Running to Stopping and then Stopped.

9. Step 5 — Create the Start Schedule

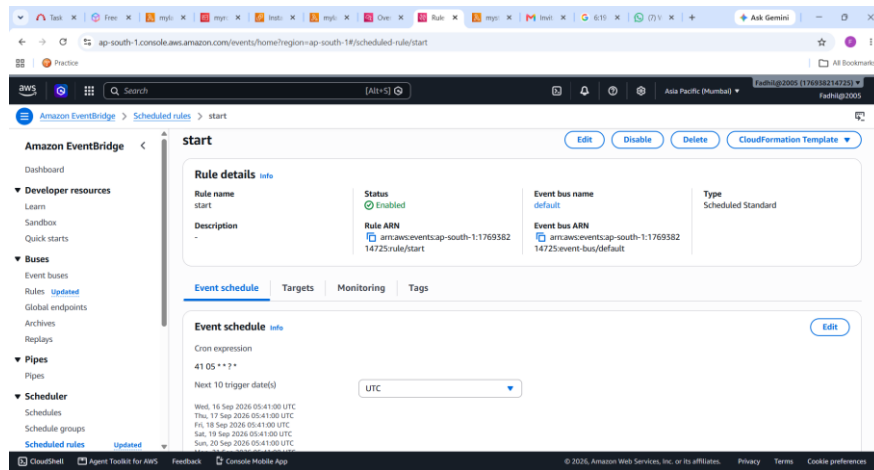
AWS now provides Amazon EventBridge for scheduled rules. In the AWS console, go to Amazon EventBridge → Rules → Create rule.

16. Rule name: EC2StartRule.
17. Rule type: Schedule.
18. Choose a recurring schedule.
19. Choose a cron-based schedule if you want a cron expression.

20. Set the target to the Lambda function Start
21. Configure the target input as JSON.

Cron time:

41 5 * * ? *



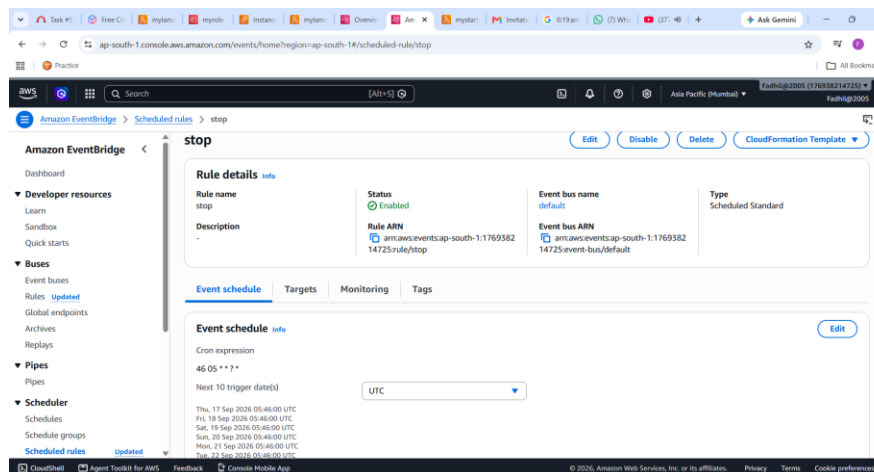
10. Create the Stop Schedule

22. Create another EventBridge rule named EC2StopRule.
23. Choose Schedule and create a recurring cron schedule.
24. Set the target to Stop
25. Configure the target input as JSON.

Cron time:

18 06 * * ? *

Modern EventBridge Scheduler supports explicit time zones, which can be useful when you want to specify schedules directly in a local time zone. Always verify the timezone configured for the scheduling mechanism you choose.



11. Final Architecture

Start: 11:11 AM → EventBridge → Lambda → EC2 Start

Stop: 11:16 AM → EventBridge → Lambda → EC2 Stop

12. Simple Explanation

IAM answers: “What is Lambda allowed to do?”

Lambda answers: “How do I start or stop the EC2 instance?”

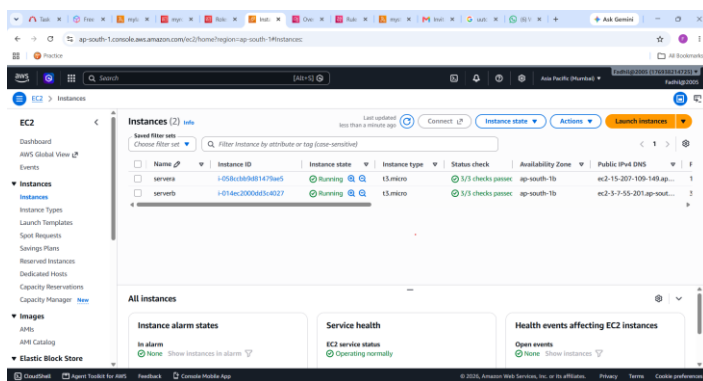
EventBridge answers: “When should Lambda run?”

EC2 is the resource that is ultimately started or stopped.

In short: EventBridge triggers Lambda at the scheduled time. Lambda uses its IAM role to call the EC2 API, and EC2 starts or stops.

OUTPUTS:

The instance start at **11:11**:



The instance stop at 11:16:

